Monthly Project Progress Report

Project Overview

During this month, the primary focus was on building and automating a complete **data annotation and object detection pipeline** — from data collection to model training, tracking, and deployment.
Key areas of progress include:

- Large-scale **annotation and model training**
- **Server setup** and **environment configuration**
- Development of **web APIs** and **live camera inference**
- **Automation of data workflows** using APIs
- Research and implementation of **automatic annotation models**

---

1. Data Preparation and Annotation Process

1.1 Initial Data Handling

- Received raw datasets (images and videos) with **inconsistent formats**.
- Reformatted and cleaned the data for compatibility with annotation tools and model training pipelines.
- Ensured directory structure and labeling consistency.

1.2 Manual Annotation

- Performed **manual annotation** for over **40 different product types**.
- Used extracted video frames to annotate product objects.

- Maintained strict data quality control to ensure accurate labeling.

1.3 Tools and Workflow

- Utilized **Roboflow** for managing datasets and annotation consistency.

Process:

Download video → Extract frames → Annotate → Upload to Roboflow

-
- Verified annotations visually and statistically to minimize labeling errors.

---

2. Server Setup and Environment Configuration

2.1 Server Initialization

- After receiving the new server, we set up:
    - **Ubuntu environment**
    - **Conda environments** for different modules (training, automation, inference)
    - Installed all required dependencies and libraries (e.g., Ultralytics, OpenCV, Flask, PyTorch).

2.2 Environment Optimization

- Configured the system for **GPU acceleration (CUDA)** to support high-speed model training.
- Verified CUDA, cuDNN, and PyTorch compatibility.
- Created separate environments for:
    - **Model training**

    - **Automation scripts**
    - **Web deployment**

---

3. Object Detection Model Development

3.1 YOLO Model Training

- Trained multiple YOLO versions to detect product objects:
    - **YOLOv8**
    - **YOLOv11 (latest version)**
- Initial training accuracy was **low** due to limited data availability.

3.2 Model Improvement

- Collected and annotated additional data to improve model performance.
- Retrained models and observed significant **accuracy and precision improvements**.
- Evaluated models on test datasets for object detection reliability.

3.3 Results

- Improved **mAP (mean average precision)** after adding new data.
- Model performance validated on unseen videos for consistency.

---

4. Product Tracking and Counting System

4.1 Pipeline Development

- Developed a **real-time tracking and counting pipeline** to track products in video streams.
- Implemented tracking algorithms:
    - **Deep SORT**
    - **ByteTrack**
- Compared tracking stability and object re-identification performance.

4.2 Testing and Evaluation

- Tested pipelines on recorded videos and live camera streams.
- Verified accurate **object count, movement tracking**, and **ID consistency**.

---

5. Web Application and API Integration

5.1 Flask-based Inference API

- Developed a **Flask web API** for testing model inference:
    - Image upload → Model inference → JSON response.
    - Later extended for **video uploads** and **real-time inference**.

5.2 Tracking Integration

- Added functionality to **run tracking models** directly through API endpoints.
- Endpoints now support:
    - `/image_inference`
    - `/video_inference`

    - `/tracking_inference`

5.3 Live Camera Streaming

- Implemented **live camera access** for real-time object detection and counting.
- Integrated **OpenCV camera stream** with model inference for live analytics.

5.4 SSL and Security Setup

- Faced blocker: live camera access was restricted due to **HTTP security policies**.
- Resolved by:
    - Installing **SSL certificates**
    - Upgrading server from **HTTP to HTTPS**
- Result: Secure live camera access and full API functionality.

---

6. Workflow Automation

6.1 Objective

To eliminate repetitive manual tasks (video download, frame extraction, upload) and streamline data preparation.

6.2 Automated Video Download

- Implemented **Google Drive API integration** to download videos automatically.
- Configured **Google Cloud Console** and generated API credentials.
- Developed a Python script with:
    - **Auto-download**

    - **Error handling**
    - **Retry mechanism** for rate limits.

6.3 Frame Extraction Automation

- Built a script to automatically **extract frames from downloaded videos**.
- Verified frame quality and extraction intervals.
- Process is now **fully automatic**.

6.4 Roboflow Upload Automation

- Automated upload of frames directly to **Roboflow** using its API.
- Replaced manual uploads, ensuring faster and more consistent dataset updates.

6.5 Unified Automated Pipeline

Combined all processes into one continuous workflow:

Download videos → Extract frames → Upload to Roboflow → Annotate/Train

---

7. Automatic Annotation Research

7.1 Objective

To reduce human effort and speed up the annotation process through AI-driven labeling.

7.2 Experiments and Findings

- **Ultralytics SAM Model**
    - Tested for segmentation-based annotation.
    - Limitation: requires pre-defined classes; not flexible for new product types.

- **CLIP Annotation Model**
    - Integrated for zero-shot annotation using text prompts.
    - Accuracy was lower than expected for product-specific cases.
- **DINO + SAM Model**
    - Current model under testing.
    - Promising results for **mask-based automatic annotation**.
    - Working on improving precision and class generalization.

---

8. Current Focus and Next Steps

8.1 Ongoing Work

- Fine-tuning **DINO + SAM** for semi-automatic annotation.
- Integrating annotation automation into the Roboflow pipeline.
- Enhancing YOLO training pipeline for higher accuracy with new data.

8.2 Upcoming Goals

- Complete automation of the **data-to-model** workflow.
- Deploy **tracking and counting system** for live environments.
- Optimize inference speed and reduce server processing latency.

Conclusion

The project made significant technical progress this month, transitioning from manual operations to a largely **automated and scalable pipeline**.

With detection, tracking, and automation all integrated, the foundation is now set for **fully intelligent annotation and real-time product monitoring**.